

# CSE 144 Final Project Report

## Transfer Learning Challenge

Group Members (max 3):  
Aden Svay (CruzID: 1974416)  
Edward Greene (CruzID: 2165823)

Spring 2026

### 1 Introduction

#### 1. Problem goal and setting.

Throughout the course of this class we learned different data handling, processing, and augmentation techniques. We also learned about different classification models and their strengths and weaknesses. Our goal with this project was to go through the entire deep learning pipeline, using data preprocessing and augmentation as well as transfer learning techniques to produce a vision classifier that would be used on a 100 class dataset.

#### 2. Why transfer learning is appropriate.

We utilize transfer learning in this assignment because there are already many very powerful image classification models trained on much larger datasets than the one given to us in this case. These models are capable of classifying images into thousands of classes, which is a larger scale version of our task. Since transfer learning is especially useful when the tasks are related, it makes it an obvious choice for this assignment.

#### 3. Brief summary of your approach and main result.

Our approach was to construct an end to end pipeline that could handle all of the necessary image processing and augmentation, plug train/val/test splits into a model of our choosing, and produce interpretable results. We repurposed our assignment 2 to achieve this, with the addition of a resizing script and different cells to handle the different models we would test for the duration of this assignment. In the end we tested ResNet (18/34/50/152), EfficientNet (B0/1/3), and ViT\_b\_16, primarily using IMAGENET1K\_V1, but also using IMAGENET1K\_V2 and IMAGENET1K\_SWAG\_E2E\_V1 where we felt they would be useful. Our main result was a local validation accuracy of 0.819 and a kaggle submission accuracy of 0.77272.

## 2 Dataset

1. Number of classes and dataset sizes (train/val/test).

We used a 2 phase unsupervised learning strategy.

Our initial training used: train = 985, val = 94, test = 1036

Once trained, we used unsupervised training with confidence  $\geq 0.7$  resulting in:

Train + unsupervised test = 1628, val = 94, test = 1036

2. Directory structure and label mapping details (ensure labels 0–99 match folders).

We used ImageFolder for the train and val sets since they were organized in class based subfolders. For our test dataset we developed a custom dataset class with a list that contained all of the ids and then used the sorted function to ensure correct mapping and ordering (avoiding the 10 before 2 etc sorting issue). Our data resizing script should sort this problem out anyway because we prepend a 0 to all single digit files to allow them to appear in order (like 09 before 10...), but with both we got the correct mapping.

3. Preprocessing (resize, normalization) and data augmentation used.

We developed a resizing script that used cv2 and glob/iter\_dir for folder traversal. In hindsight it would have been best to just use PIL for this due to the BGR flipping used by cv2, which we had to reverse using the BGR2RGB cv2 function. We resized to 224x224 for all ResNet, EffNet, and ViT (non SWAG) runs, and 384x384 for the ViT SWAG runs. Our normalization was default given by IMAGENET and we used a range of data augmentation as listed below.

RandomHorizontalFlip(p=0.5) horizontally flip half the images

ColorJitter: (0.15, 0.15, 0.15, 0.05) and (0.3, 0.3, 0.3, 0.1) eventually using latter

RandomApply(GaussianBlur(7), p=0.1) randomly lightly applied blur to 10% of images

RandomResizedCrop(224, scale=(0.6, 1.0)) ended up not using this as it was found to hurt models in our tests

RandomErasing(0.15, (0.02, 0.1)) randomly erasing small chunks of the images

MixUp functionality with very light lambda = 0.05

## 3 Implementation

### 3.1 Model

1. Pretrained backbone used (e.g., ResNet/EfficientNet/ViT) and why.

We tested ResNet18/34/50/152, EfficientNetB0/1/3, and ViT-b/16.

We ended up using the ViT-b/16 backbone because it performed the best on our tests and scored the highest on kaggle as well. We believe this is because of the models 384x384 capabilities as well as its ability to outperform ResNet and EffNet in larger scales.

2. Architecture changes (classifier head, pooling, dropout, etc.).

We changed the classifier head from 1000 classes to 100 and used 0.5 dropout.

### 3. Fine-tuning strategy (frozen layers, unfreezing schedule).

We used a phased based unfreezing strategy where we trained only the classifier head for the first 2 epochs, then the upper half (6 layers) for the next 4, then the entire model (all 12 layers) for the rest of our epochs. We did this for 2 phases due to our use of unsupervised learning, so one for the original train/val and one for (train + test unsupervised)/val

## 3.2 Training

### 1. Loss function and optimizer.

We used CrossEntropyLoss with label smoothing of 0.1 and Adam optimizer with varying decaying learning rates based on unfrozen layers.

2. Learning rate, scheduler, batch size, epochs, weight decay. 3. Hardware/software environment (GPU, PyTorch version).

Classifier head  $lr = 1e-4$ , first 6 layers  $lr = 5e-5$ , full layer  $lr = 1e-5$

Weight decay =  $1e-3$  for all

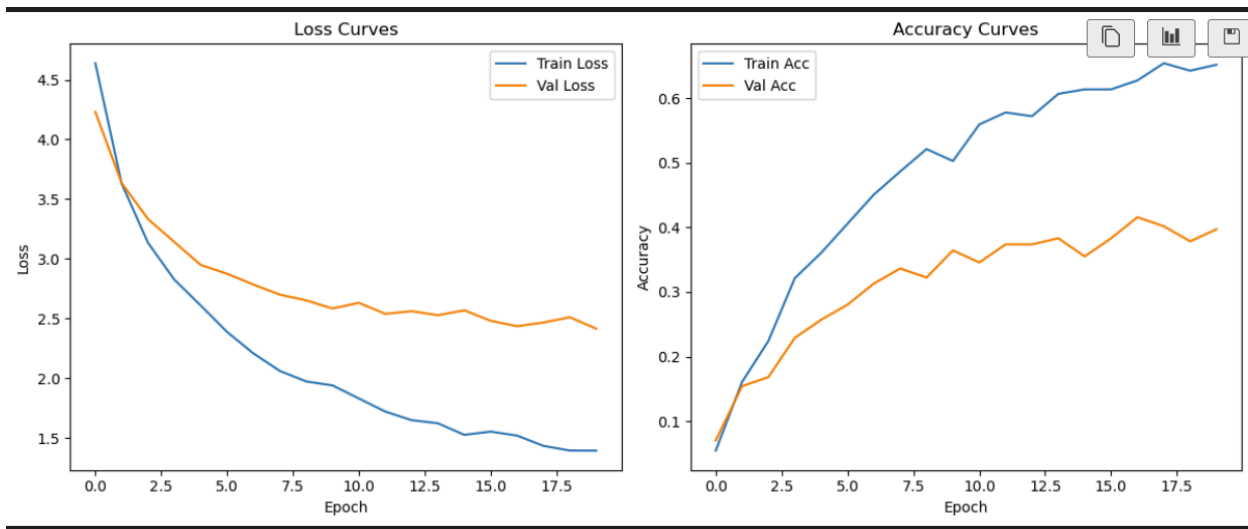
Epochs = 30 (best epoch = 19)

Trained on cuda gpu via colab, torch = 2.11.0+cu128

## 4 Experiments

### 1. Baseline setup.

Our first setup used ResNet18, HorizontalFlip and ColorJitter with the values listed above (lower levels of jitter). This model was susceptible to overfitting but was able to get a best val of  $\sim 0.41$



We would use this to test various augmentation methods that we would apply to the rest of the ResNet and EffNet runs.

## 2. Hyperparameter tuning plan and validation method.

Our hyperparameter tuning plan:

We tested various learning rates on ResNet18 to find what worked best for each layer due to the quick training results. We found that decreasing learning rate as we unfroze more of the model helped limit the amount of overfitting, which is how we landed on our tapering lr for our Adam optimizer. We found label smoothing helped, and various tests led us to 0.1 as a nice medium. Since the models always overfit, we utilized dropout to help limit this problem, eventually using 0.5 dropout, but most of our tests showed 0.3-0.5 to lead to the same results. We originally started with 0.8/0.2 train/val but found that more training images helped, so we landed on 985 as the final training value.

Validation Method:

We used the same validation script as assignment 2, noting both val loss and val acc to gauge how a model is able to translate to the test set. This was usually an accurate representation which would closely match what we got on our kaggle submissions.

## 3. Ablations (augmentations, model size, freezing strategy, LR, etc.).

Using the same ResNet18 quick training/results method we tested which data augmentations and which augmentation values produced the best results. We found that adding more augmentation often helped the model more than it hurt, however we did find RandomResizedCrop to be hurtful and therefore chose not to include it in our final train. We found that phase based unfreezing was most effective through our ResNet18 experiments but also in training our final models, so we used the unfreezing strategy mentioned above. Our learning rate tapering was found to be the best method to limit overfitting as mentioned in the hyperparameter tuning section.

## 5 Results

1. Training/validation accuracy (and loss) and any key curves/plots (optional).

The best checkpoint we used had:

Train Loss: 1.018

Train Acc: 0.515

Val Loss: 1.525

Val Acc: 0.819

This was typical of our post MixUp runs, which saw val acc above train acc but train loss above val loss. This result came from a train run with unsupervised test results used as well.

2. Kaggle public leaderboard score (and submission settings).

|    |           |                                                                                     |         |   |    |
|----|-----------|-------------------------------------------------------------------------------------|---------|---|----|
| 32 | Team Name |  | 0.77272 | 9 | 1m |
|----|-----------|-------------------------------------------------------------------------------------|---------|---|----|

 Your Best Entry!  
Your most recent submission scored 0.77272, which is an improvement over your previous score of 0.75454. Great job!

[Tweet this](#)

## 6 Discussion

1. What worked best and why.

The three biggest breakthroughs in our assignment came in the later stages of our testing. We found that using MixUp during our training helped the model generalize better, even though it caused the train to be very sporadic and not converge/improve to high 90%. This helped our validation and test accuracies to go up. We also found that

unsupervised training helped expose our model to a larger dataset than was originally given, which also allowed it to learn better. This makes sense because the larger the dataset the model is exposed to the better it will be able to do in classification when facing unseen test data. The ViT model was also the last model we tested. Specifically, ViT-b/16 with the IMAGENET1K\_SWAG\_E2E\_V1 weights and 384x384 images helped our model produce the best validation results. We believe this is because the 224x224 images were harder for the models to learn due to information being lost. We also believe that the model is simply more powerful than the Res and EffNet models we tested on before it.

## 2. Failure cases, overfitting/underfitting observations.

Our largest failure case was the EfficientNet trials in general. When compared to the ResNet and ViT runs we had, EffNet had worse validation and more overfitting, regardless of B0/1/3. We believe this is because our model was originally trained and optimized for ResNet, so the translation to EffNet was not always perfect. We also only ran EffNet on 224x224 pixel images, which may produce suboptimal results. We decided to stop trying to optimize EffNet when it consistently underperformed the other models we were using. At the beginning of our attempt at finding the best model we found significant overfitting to be the primary problem of the assignment. This is to be expected given the extremely small train dataset given to us (about 1000 images). Overfitting proved to be an insurmountable problem when only handled with data augmentation and hyperparameter tuning. When we introduced MixUp however, we found our model converged much more slowly with train loss still steadily decreasing and the train acc hovering between 0.3 to 0.6. We also found that our validation would converge to about 0.75-0.8 in most mixup cases, giving us our final result. While we never achieved underfitting, we did limit overfitting throughout the process of the assignment.

## 3. Limitations and concrete next improvements.

The biggest limitation of the project is the dataset size. Due to the small number of train (and test in the case of our unsupervised dataset) samples, our model consistently overfit the training data, and therefore could not generalize well enough to the test dataset. Our next improvement would be to use an ensemble approach to help our model probabilistically predict classes. We also briefly experimented with test time augmentation, but due to a likely incorrect setup found it to hurt our model rather than help. If we had more time we could work out a better test time augmentation script which could help rather than hurt.

# 7 Reproducibility

## 1. Random seeds and determinism settings.

We reused our assignment 2 seed and determinism, that is:

Seed = 42

Determinism:

```
torch.backends.cudnn.benchmark = False

torch.backends.cudnn.deterministic = True

torch.backends.cuda.matmul.allow_tf32 = False

torch.backends.cudnn.allow_tf32 = False

try:

    torch.use_deterministic_algorithms(True)

except Exception as e:

    print("Warning: could not enable full deterministic algorithms:",
e)
```

2. Package versions / environment setup steps.

torch: 2.11.0+cu128

torchvision: 0.26.0+cu128

numpy: 2.0.2

PIL / Pillow: 11.3.0

matplotlib: 3.10.0

sklearn: 1.6.1

pandas: 2.2.2

Python: 3.12.13

Our project is stored in an ipynb, running the first cell should be sufficient in setting up the appropriate environment.

3. Exact commands to train and to generate submission.csv. 2

Run every cell in the git repository in order

## 8 Team Contributions

1. Aden Svey: Data augmentation decisions, ResNet18/34/50/152, ViT-B/16 testing runs/documentation, image resizing script, MixUp implementation and testing

2. Ed Greene: Custom test datasetclass, EfficientNetB0/1/3, ViT-B/16, ResNet50/152 testing runs/documentation, test time augmentation testing, unsupervised learning implementation and testing

## 9 References

1. TorchVision documentation / pretrained model sources.

TorchVision: <https://docs.pytorch.org/vision/main/models.html>

<https://docs.pytorch.org/vision/main/transforms.html>

<https://docs.pytorch.org/docs/2.12/optim.html>

<https://docs.pytorch.org/docs/2.12/generated/torch.nn.CrossEntropyLoss.html>

[https://docs.pytorch.org/docs/2.12/generated/torch.optim.lr\\_scheduler.CosineAnnealingLR.html](https://docs.pytorch.org/docs/2.12/generated/torch.optim.lr_scheduler.CosineAnnealingLR.html)

We used pytorch documentation for all of the data augmentation, model, scheduler and optimizer decisions we made in this assignment

2. Any papers or tutorials used.

<https://www.geeksforgeeks.org/python/image-resizing-using-opencv-python/>

<https://www.geeksforgeeks.org/python/python-loop-through-folders-and-files-in-directory/>

[https://docs.pytorch.org/tutorials/beginner/basics/data\\_tutorial.html](https://docs.pytorch.org/tutorials/beginner/basics/data_tutorial.html)

[https://docs.pytorch.org/tutorials/beginner/transfer\\_learning\\_tutorial.html](https://docs.pytorch.org/tutorials/beginner/transfer_learning_tutorial.html)

Above are the transfer learning, image resizing, file iteration, and dataset creation tutorials used for our assignment.